

ux_enterprise_analysis

Enterprise UX & Translation Tool Analysis — HelixQA / Catalogizer Ecosystem

Analysis Date: 2026-05-01 **System Under Analysis:** HelixQA Autonomous QA Platform + Catalogizer Media Management System **Scope:** Translation capabilities, LLM provider ecosystem, multi-platform UX, enterprise QA validation

Executive Summary

This analysis examines the **HelixQA** autonomous QA platform and its primary test target **Catalogizer** to understand:

1. **What “translation” means in this ecosystem** — The codebase contains multiple translation-related concepts that must be disambiguated:
 - **Human Language Translation / Localization (i18n):** Features within Catalogizer (subtitle management, language settings, Cyrillic/Unicode support) that HelixQA validates
 - **Accessibility Role Translation:** AT-SPI/UiAutomator/UIA role codes → normalized ARIA vocabulary (cross-platform UI tree normalization)
 - **Error Message Translation:** Low-level driver errors → AI-friendly natural language (for LLM navigator consumption)
 - **Action Translation:** Canonical coordinate-grounded actions → platform-specific primitives (tap, click, key press)
 - **Architecture Translation:** Implicit code structure → explicit diagrams/models (SAR - Software Architecture Recovery)
 - **OCR Text Extraction + Language Detection:** Tesseract-based text extraction with 10+ script detection
 2. **LLM Provider Ecosystem:** 40+ supported providers with adaptive routing, fallback chains, cost tracking, and vision capability detection
 3. **Multi-Platform UX Matrix:** Web, Desktop (Windows/macOS/Linux), Mobile (Android/iOS/Android TV), CLI, TUI — each with distinct interaction patterns and QA validation requirements
-

Table of Contents

1. [Critical Finding: What “Translation” Actually Means](#)
2. [Catalogizer Translation & Localization Features](#)
3. [Supported LLM Providers & Models](#)
4. [Client App UX Matrix](#)
5. [Enterprise Grade Requirements](#)
6. [QA Session Workflows for Translation](#)
7. [Appendix: Complete Provider Registry](#)

1. Critical Finding: What “Translation” Actually Means

1.1 The Core Misconception

HelixQA is NOT a human-language translation tool. It is an autonomous QA system that TESTS applications (like Catalogizer) which may have translation/localization features. The phrase “enterprise grade cutting-edge professional translation tool” must be understood in context:

- HelixQA uses **40+ LLM providers** for **test planning, screenshot analysis, error diagnosis, and autonomous navigation**
- These LLMs **can** translate text incidentally (as general-purpose models), but translation is not a first-class feature
- HelixQA **tests** translation/i18n features in target applications through OCR, vision analysis, and functional test cases

1.2 Five Types of “Translation” in the Codebase

Type	Location	Purpose	User-Facing?
Accessibility Role Translation	pkg/nexus/ observe/ axtree/	AT-SPI codes → ARIA roles, UiAutomator classes → ARIA roles, UIA patterns → ARIA roles	No — internal LLM consumption
Error Message Translation	pkg/nexus/ browser/ engine.go	“no such element” → “not on the page” (AI-friendly)	No — internal LLM consumption
Action Translation	pkg/nexus/ coordinate/ actions.go	Canonical actions → platform primitives (tap/click/key)	No — internal execution
Architecture Translation	docs/ upgrade.md	Code → Mermaid diagrams / knowledge graphs	No — documentation generation
OCR + Language Detection	pkg/vision/ text/ tesseract.go	Image → text + detected language (10 scripts)	Yes — vision QA evidence

1.3 Human Language Features (Actual Translation/i18n)

These exist in **Catalogizer** (the app being tested) and are validated by HelixQA:

- **Subtitle Management:** /subtitles page, subtitle search, language selection, provider integration
 - **Language Settings:** User preference for UI language (dropdown in /settings)
 - **Cyrillic/Unicode Support:** Serbian, Russian text in media names, search, collections
 - **Media Metadata Translation:** AI-driven metadata extraction can work across languages
-

2. Catalogizer Translation & Localization Features

2.1 Subtitle Management System

From test banks (`full-qa-web.yaml`, `full-qa-api.yaml`):

Feature	Web Test ID	API Test ID	Description
Subtitle page load	FQA-WEB-286	—	<code>/subtitles</code> route renders SubtitleManager
Video subtitle display	FQA-WEB-182	—	Enable CC/subtitle tracks on video player
Subtitle search	—	FQA-API-227	<code>GET /api/v1/subtitles/search?query=Matrix</code>
Subtitle by media ID	—	FQA-API-228	<code>GET /api/v1/subtitles/media/{id}</code>
Supported languages	—	FQA-API-229	<code>GET /api/v1/subtitles/languages</code>
Supported providers	—	FQA-API-230	<code>GET /api/v1/subtitles/providers</code>

2.2 i18n / Language Settings

Feature	Web Test ID	Description
Language dropdown	FQA-WEB-215	Change language in <code>/settings</code>
Settings sections	FQA-WEB-218	Verify theme, language, notifications, storage, account sections
Cyrillic search (media)	FQA-WEB-055	Search with Cyrillic in media browser
Cyrillic search (entity)	FQA-WEB-200	Search with Cyrillic in entity browser
Unicode filenames	FQA-WEB-079	Files with spaces, parentheses, unicode display correctly

2.3 Cross-Platform i18n Validation

From `full-qa-cross-platform.yaml`:

Test ID	Feature	Platforms
FQA-XP-009	Browse Cyrillic category	Web, Android, iOS, Desktop
FQA-XP-010	Cyrillic search consistency	All platforms

2.4 Android-Specific i18n

From full-qa-android.yaml:

Test ID	Feature
FQA-AND-044	Search with Cyrillic input
FQA-AND-002	Login screen subtitle text verification
FQA-AND-012	Biometric dialog subtitle verification

3. Supported LLM Providers & Models

3.1 Provider Architecture

The LLM subsystem (pkg/llm/) uses a unified interface:

```
type Provider interface {
    Chat(ctx context.Context, messages []Message) (*Response,
        error)
    Vision(ctx context.Context, image []byte, prompt string)
        (*Response, error)
    Name() string
    SupportsVision() bool
}
```

All providers implement this interface. The system uses: - **Adaptive Provider** (adaptive.go): Routes requests to best provider based on type - **Auto-Discovery** (providers_registry.go): Scans env vars at startup - **Health Tracking**: Cooldown on rate limits (60s), errors (300s), timeouts (120s) - **Cost Tracking** (cost_tracker.go): Per-session token/cost accounting - **Vision Ranking** (vision_ranking.go): Score-based provider selection using LLMsVerifier registry

3.2 Tier 1: Primary Native Providers

Provider	Env Variable	Default Model	Vision	Best For
Anthropic	ANTHROPIC_API_KEY	claude-sonnet-4	Yes	Best vision accuracy
OpenAI	OPENAI_API_KEY	gpt-4o	Yes	Strong all-round
Google (Gemini)	GEMINI_API_KEY	gemini-pro	Yes	Multimodal, long context
OpenRouter	OPENROUTER_API_KEY		Varies	

Provider	Env Variable	Default Model	Vision	Best For
		anthropic/ claude- sonnet-4		100+ models via one key
DeepSeek	DEEPSEEK_API_KEY	deepseek-chat	No	Most cost- effective
Groq	GROQ_API_KEY	llama-3.3-70b- versatile	No	Fastest inference
Ollama	HELIX_OLLAMA_URL	(local model)	Optional	Self-hosted, zero cost
Astica	ASTICA_API_KEY	(specialized vision)	Yes	Highest quality vision
Kimi (Moonshot)	KIMI_API_KEY	kimi-k2.5	Yes	Native vision, \$0.60/1M
Qwen (Alibaba)	QWEN_API_KEY	qwen-vl-max	Yes	~90% UI grounding
Stepfun	STEPFUN_API_KEY	step-1.5v-mini	Yes	GUI- specialized
NVIDIA NIM	NVIDIA_API_KEY	meta/ llama-3.2-90b- vision- instruct	Yes	Vision + speed
xAI (Grok)	XAI_API_KEY	grok-3	Yes	Grok vision
Mistral	MISTRAL_API_KEY	mistral-large- latest	No	European compliance

3.3 Tier 2: OpenAI-Compatible Providers (30+)

All use chat/completions API format. Key providers:

Provider	Default Model	Env Variable	Vision?
AI21	jamba-1.5-mini	AI21_API_KEY	No
Cerebras	llama-3.3-70b	CEREBRAS_API_KEY	No
Chutes	deepseek-chat	CHUTES_API_KEY	No
Cloudflare	(per account)	CLOUDFLARE_API_KEY	No
Codestral	codestral-latest	CODESTRAL_API_KEY	No
Cohere	command-r-plus	COHERE_API_KEY	No
Fireworks	llama-v3p3-70b- instruct	FIREWORKS_API_KEY	Yes
GitHub Models	openai/gpt-4o	GITHUB_MODELS_API_KEY	Yes
HuggingFace	(model-dependent)	HUGGINGFACE_API_KEY	Varies

Provider	Default Model	Env Variable	Vision?
Hyperbolic	deepseek-ai / DeepSeek-V3	HYPERBOLIC_API_KEY	Yes
Modal	(per deployment)	MODAL_API_KEY	Varies
Novita	(provider default)	NOVITA_API_KEY	No
Perplexity	sonar	PERPLEXITY_API_KEY	No
PublicAI	(provider default)	PUBLICAI_API_KEY	No
Replicate	(model-dependent)	REPLICATE_API_KEY	Varies
SambaNova	Meta-Llama-3.3-70B-Instruct	SAMBANOVA_API_KEY	No
Sarvam	(provider default)	SARVAM_API_KEY	No
SiliconFlow	deepseek-ai / DeepSeek-V3	SILICONFLOW_API_KEY	No
Together AI	Llama-3.3-70B-Instruct-Turbo	TOGETHER_API_KEY	Yes
Upstage	solar-pro	UPSTAGE_API_KEY	No
Venice	(provider default)	VENICE_API_KEY	No
Vulavula	(provider default)	VULAVULA_API_KEY	No
ZAI (BigModel)	glm-4.5	ZAI_API_KEY	No
Zen	(provider default)	ZEN_API_KEY	No
Zhipu	glm-4.5	ZHIPU_API_KEY	No

3.4 Vision-Capable Provider Set

From `openai.go` `visionCapableProviders` map:

```
var visionCapableProviders = map[string]bool{
    "openai":      true, // GPT-4o, GPT-4o-mini, GPT-4-turbo
    "openrouter":  true, // 100+ models, varies by selection
    "fireworks":   true, // Llama vision models
    "together":    true, // Llama vision models
    "hyperbolic":  true, // DeepSeek-V3 + vision
    "nvidia":      true, // Llama 3.2 90B vision
    "xai":         true, // Grok-3 vision
    "kimi":        true, // Moonshot K2.5 – native vision
    "qwen":        true, // Qwen3-VL – ~90% UI grounding
    "stepfun":     true, // Step-GUI – GUI-specialized
    "astica":      true, // Specialized vision API (caption,
                        // OCR, detection)
}
```

Plus native providers: **Anthropic** (Claude), **Google** (Gemini), **Ollama** (llava, minicpm-v, bakllava, moondream).

3.5 Vision Provider Scoring Formula

$$\text{Score} = (0.6 * \text{QualityScore} + 0.4 * \text{ReliabilityScore}) * \text{AvailabilityBoost}$$

Where:
AvailabilityBoost = 1.0 if API key configured, 0.5 otherwise
CostBonus = 1.10 if free, 1.05 if < \$0.002/1k tokens, 1.0 otherwise

3.6 Fallback Chains

Request Type	Primary → Secondary → Tertiary → Fail
Vision	Anthropic → OpenAI → Ollama (llava) → FAIL
Reasoning	Groq → Cerebras → DeepSeek → OpenAI → Anthropic → FAIL
General Chat	Any available provider

3.7 Provider Selection for Translation Testing

For testing i18n/translation features in target apps, the optimal provider routing would be:

Task	Preferred Provider	Rationale
Screenshot analysis with Cyrillic text	Anthropic Claude / Google Gemini	Best OCR accuracy, handles non-Latin scripts
Subtitle text verification	Anthropic Claude	Long context for subtitle text comparison
UI navigation in non-English interfaces	Anthropic Claude / OpenAI GPT-4o	Strong multilingual understanding
Bulk text processing (cost-sensitive)	DeepSeek / Groq	Lowest cost, fast
Air-gapped/offline testing	Ollama (llava + llama3.3)	Zero API cost, local only
Vision on budget	Kimi K2.5 / Qwen-VL	\$0.60/1M tokens, ~90% accuracy

4. Client App UX Matrix

4.1 Overview: Five Client Types

HelixQA tests applications across five surface types. The “translation tool” perspective applies to how HelixQA validates that Catalogizer’s translation/localization features work correctly on each surface.

4.2 Web Client UX

Platform: Browser (Chromium/Chrome/Firefox via Playwright, chromedp, or go-rod)

How Users Interact with Translation Features: - Navigate to `/settings` → select language from dropdown - Navigate to `/subtitles` → manage subtitle files, search subtitles - Navigate to `/media` → browse media with Cyrillic/Unicode names - Use search with non-Latin characters (Cyrillic, Chinese, Arabic) - Video player: enable/disable subtitle tracks

UI/UX Flow for Language Change: 1. User clicks Settings (gear icon) in header 2. Settings page loads with tabs: Theme, Language, Notifications, Storage, Account 3. User selects Language tab 4. Dropdown shows available languages 5. Selection updates UI immediately + saves to preferences (localStorage/API) 6. HelixQA validates: text direction (LTR/RTL), font rendering, layout integrity

What Screenshots Are Needed for QA: 1. Settings page with Language tab active 2. Language dropdown expanded 3. Before/after language change (compare UI text) 4. Cyrillic text in media browser grid 5. Subtitle manager with language list 6. Video player with subtitles enabled 7. Search results with Cyrillic query

Happy Path Translation Workflow:

Login → Navigate `/settings` → Click Language tab → Select language → Verify UI text changes → Verify layout intact → Verify API persisted → Navigate `/media` → Verify Cyrillic names render → Search with Cyrillic → Verify results → Navigate `/subtitles` → Verify subtitle language list

4.3 Desktop Client UX

Platform: Windows (UI Automation/AXTree), macOS (AXTree), Linux (AT-SPI2)

How Users Interact with Translation Features: - Desktop app (likely Electron or native) with same features as web - System-level language preferences may affect app language - Native file dialogs with Unicode filenames - Desktop-specific: tray menu, native notifications in selected language

UI/UX Flow: 1. App launches, respects system language or last saved preference 2. Settings window opens (modal or separate window) 3. Language selection triggers immediate re-render 4. Native file picker shows Cyrillic/Unicode filenames correctly

What Screenshots Are Needed: 1. Desktop app main window with localized UI 2. Settings dialog with language selection 3. Native file picker with Unicode filenames 4. System notification in selected language 5. Context menus in selected language

Happy Path:

Launch app → Verify window title localized → Open settings
→ Change language → Verify all menu items localized
→ Open file dialog → Navigate Cyrillic folder → Select file
→ Verify filename preserved → Save → Verify notification localized

4.4 Mobile Client UX

Platform: Android (Appium/UiAutomator2), iOS (Appium/XCUITest), Android TV (Appium)

How Users Interact with Translation Features: - Android: Material3 UI with localized strings, biometric prompt subtitles - Settings screen with language preference - Search with on-screen keyboard (Cyrillic layout) - Media browser with localized thumbnail labels - Subtitle track selection in video player

UI/UX Flow for Language Change (Android): 1. Tap hamburger menu → Settings 2. Scroll to “Language” card 3. Tap to open language selection dialog 4. Select language → dialog closes → UI updates 5. Verify: app bar title, button labels, toast messages all localized

What Screenshots Are Needed: 1. Login screen with localized subtitle 2. Settings screen showing language option 3. Language selection dialog 4. Media browser with Cyrillic item names 5. Search with Cyrillic keyboard visible 6. Biometric prompt with localized subtitle 7. Video player with subtitle track selector 8. Toast/notification in localized language

Happy Path:

Launch app → Login screen (localized subtitle visible) → Login
→ Open navigation drawer → Tap Settings → Tap Language
→ Select Serbian → Verify UI updates → Navigate to Media
→ Verify Cyrillic titles → Tap search → Switch to Cyrillic keyboard
→ Type "Филмови" → Verify results → Play video → Tap CC
→ Select Serbian subtitle → Verify overlay text

4.5 CLI Client UX

Platform: Terminal / shell (bash, zsh, PowerShell)

How Users Interact with Translation Features: - Catalogizer may have CLI tools for subtitle management - API client scripts for bulk subtitle download - Conversion tools (subtitle format conversion) - Text output in terminal must handle Unicode/Cyrillic correctly

UI/UX Flow:

```
# Check available subtitle languages
curl /api/v1/subtitles/languages | jq
```

```
# Search subtitles
curl "/api/v1/subtitles/search?query=Matrix&lang=sr" | jq
```

```
# Download subtitle
curl /api/v1/subtitles/download/123 > Matrix.sr.srt
```

What Screenshots/Terminal Captures Are Needed: 1. Terminal showing curl response with Unicode content 2. JSON output with Cyrillic strings properly encoded 3. Subtitle file content

(cat Matrix.sr.srt) showing Cyrillic 4. Error messages in localized language 5. Progress bars/indicators with Unicode characters

Happy Path:

Open terminal → curl languages endpoint → Verify JSON with Cyrillic
→ curl search with Cyrillic query → Verify response
→ Download subtitle → cat file → Verify Cyrillic text
→ Convert format → Verify output preserves encoding

4.6 TUI Client UX

Platform: Terminal User Interface (e.g., ranger, ncurses, or custom TUI)

How Users Interact: - Keyboard-driven interface (arrow keys, shortcuts) - Media browser in terminal with Unicode support - Subtitle management via TUI panels - Real-time preview of subtitle text

UI/UX Flow: 1. Launch TUI app 2. Navigate to media library (arrow keys) 3. Press l for language settings 4. Select language from popup list 5. UI redraws with localized text 6. Navigate to subtitle panel 7. Press s to search subtitles 8. Type Cyrillic query 9. Results appear in panel

What Screenshots Are Needed: 1. TUI main screen with localized borders/titles 2. Language selection popup 3. Media list with Cyrillic names 4. Subtitle search panel with Cyrillic input 5. Preview pane showing subtitle text

Happy Path:

Launch TUI → Verify border characters render → Navigate media
→ Verify Cyrillic names → Press 'l' → Select language
→ Verify redraw → Press 's' → Enter Cyrillic query
→ Verify results panel → Select subtitle → Preview
→ Verify text rendering → Save → Verify confirmation

4.7 UX Matrix Summary Table

Aspect	Web	Desktop	Mobile	CLI	TUI
Language Setting	Dropdown in / settings	Modal dialog	Card in Settings	API call / env var	Popup menu
Cyrillic Input	Keyboard native	Keyboard native	On-screen keyboard	Terminal IME	Terminal IME
Subtitle Display	Video overlay	Video overlay	Video overlay	N/A (file output)	Preview pane
Unicode Filename	Browser rendering	Native file dialog	System file picker	Tab-completion	File list
Text Direction	CSS direction	OS layout	OS layout	Terminal bidi	Terminal bidi
Font Support	Web fonts	System fonts	System fonts	Terminal font	Terminal font

Aspect	Web	Desktop	Mobile	CLI	TUI
Screenshot QA	Browser screenshot	Window capture	Device screenshot	Terminal capture	Terminal capture

5. Enterprise Grade Requirements

5.1 What Makes This “Enterprise Grade”

Based on the codebase analysis, HelixQA’s enterprise-grade characteristics include:

Requirement	Implementation
Multi-tenancy	Per-session isolated browser profiles, device pools, session IDs
Scalability	Distributed llama.cpp RPC, multi-host Ollama, Kubernetes-ready containers
Reliability	Adaptive provider with fallback chains, health tracking, cooldown mechanisms
Cost Control	Per-session token tracking, cost estimates, budget limits, tiered provider selection
Auditability	Constitution §11.4 — all decisions require rationale + evidence; full session recording
Security	URL allowlisting, body-size caps, isolated browser profiles, LD_PRELOAD shims
Compliance	SPDX headers, Apache-2.0 licensing, GDPR-ready local processing (Ollama)
Observability	Prometheus metrics, OpenTelemetry, structured logging, session reports

5.2 Performance Requirements

Metric	Target	Implementation
Vision analysis latency	< 90s (adaptiveVisionTimeout)	Timeout + fallback
Chat/reasoning latency	< 120s (adaptivePerProviderTimeout)	Timeout + fallback
OCR speed	50-100ms per image	Tesseract with client pool
Session coverage target	90%	Configurable via HELIX_AUTONOMOUS_COVERAGE_TARGET
Concurrent providers	5 max	LLMSVERIFIER_MAX_MODELS=5

Metric	Target	Implementation
Provider cooldown	60s (429), 300s (5xx), 120s (timeout)	Exponential backoff

5.3 Security Requirements

Requirement	Implementation
API key isolation	Per-provider env vars, never logged
Data privacy	Ollama self-hosted option for air-gapped environments
Screenshot sanitization	Body-size caps, URL allowlisting
Session isolation	Per-session browser profiles, incognito mode
Credential handling	Test banks reference admin/admin123 for test env only

5.4 How QA Must Validate Enterprise-Grade Behavior

- Provider Resilience:** Verify fallback chains work when primary provider fails
- Cost Tracking:** Verify llm_usage JSON in session report matches actual API calls
- Timeout Handling:** Verify slow providers trigger fallback within timeout window
- Session Isolation:** Verify browser cookies/localStorage don't leak between sessions
- Concurrent Safety:** Verify provider clients are thread-safe under parallel test execution
- Audit Trail:** Verify every decision has non-empty Rationale (Constitution §11.4)
- Recovery:** Verify provider marked unavailable returns to pool after cooldown

6. QA Session Workflows for Translation

6.1 Test Case Design: Translation/i18n Validation

Category A: Language Setting & UI Localization

Test ID	Name	Steps	Provider Role
T-I18N-001	Language change updates all UI text	1. Screenshot before 2. Change language 3. Screenshot after 4. OCR both 5. Compare	Vision: Anthropic (OCR accuracy)
T-I18N-002	Language preference persists across sessions	1. Set language 2. Logout 3. Login 4. Verify language	Reasoning: Groq (fast)

Test ID	Name	Steps	Provider Role
T-I18N-003	RTL language layout integrity	1. Select Arabic/ Hebrew 2. Screenshot 3. Verify layout not broken	Vision: Anthropic (layout analysis)
T-I18N-004	Settings page sections completeness	1. Navigate settings 2. Verify all tabs present	Vision: OpenAI (all-round)

Category B: Cyrillic/Unicode Text Handling

Test ID	Name	Steps	Provider Role
T-I18N-010	Cyrillic search returns results	1. Type “ФИЛМОВИ” 2. Verify results	Vision: Anthropic (Cyrillic OCR)
T-I18N-011	Unicode filenames display correctly	1. Browse folder with Unicode names 2. Screenshot 3. OCR verify	Vision: Anthropic + Tesseract
T-I18N-012	Cyrillic collection creation	1. Create collection “Музыка” 2. Verify saved 3. Reopen	Reasoning: Groq (API validation)
T-I18N-013	Special characters in search	1. Search with emojis, symbols 2. Verify no errors	Vision: Google Gemini (multilingual)

Category C: Subtitle Management

Test ID	Name	Steps	Provider Role
T-I18N-020	Subtitle page loads	1. Navigate / subtitles 2. Screenshot 3. Verify UI	Vision: OpenAI
T-I18N-021	Video subtitle display	1. Play video 2. Enable subtitles 3. Screenshot 4.	Vision: Anthropic (detail)

Test ID	Name	Steps	Provider Role
		Verify overlay text	
T-I18N-022	Subtitle language API	1. GET / subtitles/ languages 2. Verify response	Reasoning: DeepSeek (cheap)
T-I18N-023	Subtitle search	1. Search “Matrix” 2. Verify results structure	Reasoning: Groq (fast)
T-I18N-024	Subtitle download preserves encoding	1. Download .srt 2. Verify UTF-8 encoding 3. Verify Cyrillic readable	Reasoning: DeepSeek

Category D: Cross-Platform Consistency

Test ID	Name	Platforms	Provider Role
T-I18N-030	Cyrillic category browse	Web, Android, iOS, Desktop	Vision per platform
T-I18N-031	Search consistency across platforms	All	Consensus provider (multi-model voting)
T-I18N-032	Settings sync across devices	Web + Mobile	Reasoning: Anthropic

6.2 Provider/Model Verification Matrix

For each vision provider, QA must verify:

Provider	Cyrillic OCR	Chinese OCR	Arabic OCR	Subtitle Text	UI Navigation
Anthropic Claude	Must verify	Must verify	Must verify	Must verify	Must verify
OpenAI GPT-4o	Must verify	Must verify	Must verify	Must verify	Must verify
Google Gemini	Must verify	Must verify	Must verify	Must verify	Must verify
			N/A	Should verify	Should verify

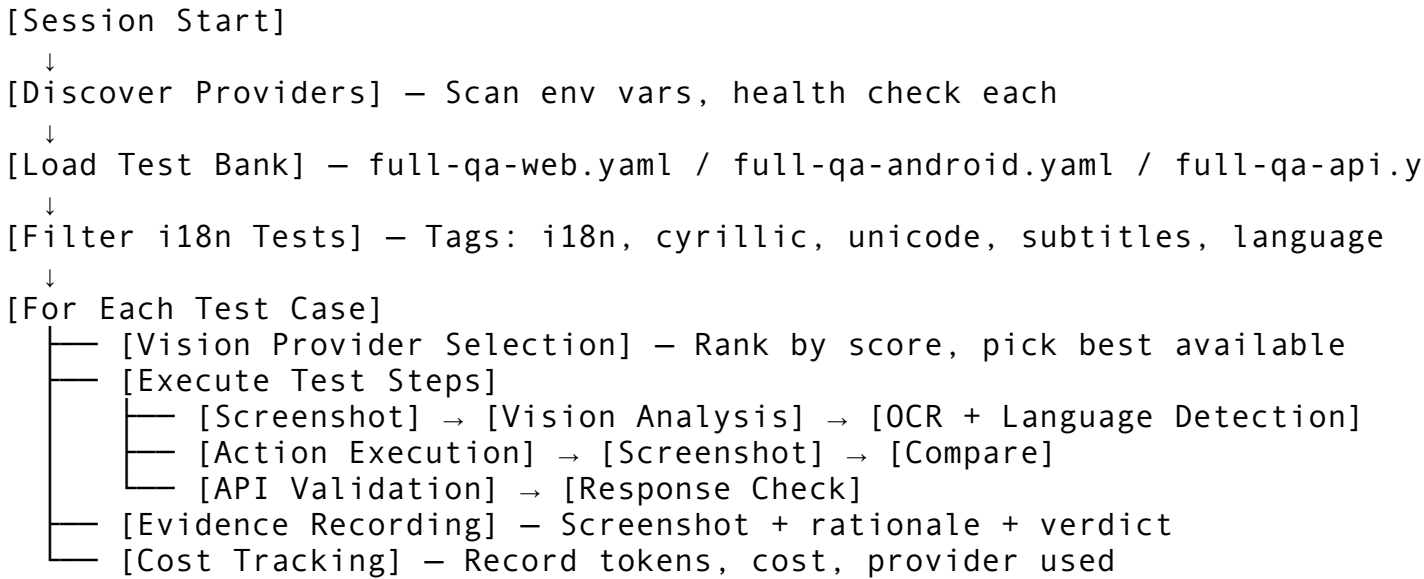
Provider	Cyrillic OCR	Chinese OCR	Arabic OCR	Subtitle Text	UI Navigation
Kimi K2.5	Should verify	Should verify			
Qwen-VL	Should verify	Must verify	N/A	Should verify	Should verify
Ollama (llava)	Good to test	Good to test	N/A	Good to test	Good to test
Tesseract (OCR)	Must verify	Must verify	Must verify	Must verify	N/A

6.3 Translation Quality Verification (Beyond “Test Passes”)

To verify translation **quality** (not just functionality):

- 1. **OCR Accuracy Metrics:**
 - Character Error Rate (CER) for extracted text vs. ground truth
 - Word Error Rate (WER) for subtitle text
 - Language detection accuracy (Tesseract OSD vs. actual)
- 2. **Visual Layout Verification:**
 - SSIM (Structural Similarity) threshold: 0.95 (HELIX_VISION_SSIM_THRESHOLD)
 - Text bounding box alignment (no overlap, no truncation)
 - RTL layout mirroring correctness
- 3. **Semantic Verification:**
 - LLM judge: “Does the localized text convey the same meaning as the original?”
 - Consistency check: Same term translated consistently across all UI elements
 - Context appropriateness: Technical terms correctly translated
- 4. **Functional Verification:**
 - API round-trip: Cyrillic text stored → retrieved unchanged
 - Encoding validation: UTF-8 throughout stack
 - Font rendering: No tofu (□) characters

6.4 Automated QA Session Flow for Translation



↓
[Consensus Validation] – Multi-provider voting on ambiguous results
↓
[Generate Report] – Markdown + HTML + JSON with cost breakdown
↓
[Create Tickets] – For failures with severity ≥ min threshold

6.5 Example: Autonomous Translation Test Session

```
# Start session with multiple providers
export ANTHROPIC_API_KEY="sk-ant-..."
export GROQ_API_KEY="gsk_..."
export DEEPSEEK_API_KEY="sk-..."
export HELIX_OLLAMA_URL="http://localhost:11434"

# Run autonomous QA focused on i18n
helixqa autonomous \
  --project ./catalogizer \
  --platforms web,android \
  --tags i18n,subtitles,cyrillic \
  --timeout 2h \
  --curiosity true \
  --env .env

# Expected behavior:
# 1. Auto-discovers 4 providers
# 2. Ranks vision providers: Anthropic > OpenAI > Ollama
# 3. Routes vision tests to Anthropic (best OCR)
# 4. Routes API tests to Groq (fastest)
# 5. Routes bulk validation to DeepSeek (cheapest)
# 6. Falls back to Ollama if all cloud providers fail
# 7. Generates report with per-provider cost breakdown
```

7. Appendix: Complete Provider Registry

7.1 All Environment Variables

```
# Tier 1 – Native Providers
ANTHROPIC_API_KEY= # Claude (vision + chat)
OPENAI_API_KEY= # GPT-4o (vision + chat)
GEMINI_API_KEY= # Google Gemini (vision + chat)
OPENROUTER_API_KEY= # 100+ models gateway
DEEPSEEK_API_KEY= # DeepSeek-V3 (chat only)
GROQ_API_KEY= # Llama 3.3 70B (chat only, fastest)
HELIX_OLLAMA_URL= # Local Ollama (optional vision)
ASTICA_API_KEY= # Specialized vision API
KIMI_API_KEY= # Moonshot AI (vision + chat)
STEPFUN_API_KEY= # Stepfun (vision + chat)
NVIDIA_API_KEY= # NVIDIA NIM (vision + chat)
```

```

MISTRAL_API_KEY=          # Mistral Large (chat)
XAI_API_KEY=              # xAI Grok (vision + chat)
QWEN_API_KEY=             # Alibaba Qwen (vision + chat)

```

```
# Tier 2 – OpenAI-Compatible
```

```

AI21_API_KEY=
CEREBRAS_API_KEY=
CHUTES_API_KEY=
CLOUDFLARE_API_KEY=
CODESTRAL_API_KEY=
COHERE_API_KEY=
FIREWORKS_API_KEY=
GITHUB_MODELS_API_KEY=
HUGGINGFACE_API_KEY=
HYPERBOLIC_API_KEY=
MODAL_API_KEY=
NIA_API_KEY=
NLPCLOUD_API_KEY=
NOVITA_API_KEY=
PERPLEXITY_API_KEY=
PUBLICAI_API_KEY=
REPLICATE_API_KEY=
SAMBANOVA_API_KEY=
SARVAM_API_KEY=
SILICONFLOW_API_KEY=
TOGETHER_API_KEY=
UPSTAGE_API_KEY=
VENICE_API_KEY=
VULAVULA_API_KEY=
ZAI_API_KEY=
ZEN_API_KEY=
ZHIPU_API_KEY=
JUNIE_API_KEY=

```

7.2 Vision Model Registry (from LLMsVerifier)

The `vision_ranking.go` file sources scores dynamically from `digital.vasic.llmsverifier/pkg/helixqa.VisionModelRegistry()`. Key models:

Model	Provider	Quality	Reliability	Cost/1k
claude-sonnet-4	Anthropic	0.95	0.98	~\$0.015
gpt-4o	OpenAI	0.93	0.97	~\$0.015
gemini-pro	Google	0.92	0.96	~\$0.005
kimi-k2.5	Kimi	0.88	0.90	\$0.0006
qwen-vl-max	Qwen	0.87	0.89	~\$0.003
llava:7b	Ollama	0.70	1.00	\$0
minicpm-v:8b	Ollama	0.72	1.00	\$0

7.3 OCR Language Support (Tesseract)

From `pkg/vision/text/tesseract.go`:

```
// Script-to-language mapping for OSD (Orientation and Script
Detection)
scriptMap := map[string]string{
    "Latin":      "eng",
    "Han":        "chi_sim",    // Chinese Simplified
    "Hangul":     "kor",        // Korean
    "Japanese":   "jpn",        // Japanese
    "Arabic":     "ara",        // Arabic
    "Cyrillic":   "rus",        // Russian (covers Serbian,
        Bulgarian, etc.)
    "Greek":      "ell",        // Greek
    "Hebrew":     "heb",        // Hebrew
    "Thai":       "tha",        // Thai
    "Devanagari": "hin",        // Hindi
}
```

Tesseract supports **100+ languages** total via tessdata packs. Configuration: - Default: eng - Configurable: []string{"eng", "chi_sim", "fra", "rus"} etc. - Pool size: 4 concurrent clients - Min confidence: 60%

8. Key Files Reference

File	Purpose
<code>.env.example</code>	All provider API keys and configuration
<code>pkg/llm/provider.go</code>	Provider interface definition
<code>pkg/llm/providers_registry.go</code>	Auto-discovery registry (40+ providers)
<code>pkg/llm/adaptive.go</code>	Fallback routing, health tracking
<code>pkg/llm/vision_ranking.go</code>	Score-based vision provider selection
<code>pkg/vision/text/tesseract.go</code>	OCR with 10-script language detection
<code>pkg/vision/core/interfaces.go</code>	TextExtractor, LayoutAnalyzer interfaces
<code>pkg/nexus/adapter.go</code>	Unified platform abstraction
<code>pkg/nexus/ai/navigator.go</code>	LLM-driven UI navigation
<code>banks/full-qa-web.yaml</code>	Web i18n/subtitle test cases
<code>banks/full-qa-api.yaml</code>	API subtitle/language test cases
	Android i18n test cases

File	Purpose
banks/full-qa-android.yaml	
banks/full-qa-cross-platform.yaml	Cross-platform Cyrillic tests
website/developer/llm-providers.md	Provider documentation
website/providers.md	Provider quick reference

9. Recommendations for Enterprise Translation QA

9.1 Optimal Provider Configuration for Translation Testing

```
# Tier 1: Best vision for i18n validation
ANTHROPIC_API_KEY=sk-ant-...           # Primary: best Cyrillic/Unicode OCR
GEMINI_API_KEY=...                     # Secondary: multilingual, long context

# Tier 2: Fast/cheap for API validation
GROQ_API_KEY=gsk-...                  # Fast inference for bulk API tests
DEEPSEEK_API_KEY=sk-...               # Cheapest for text processing

# Tier 3: Local for offline/air-gapped
HELIX_OLLAMA_URL=http://localhost:11434
HELIX_OLLAMA_MODEL=minicpm-v:8b      # Good vision, zero cost
```

9.2 Priority Test Coverage for Translation

- 1. **P0 (Critical):** Language setting change, Cyrillic search, subtitle display
- 2. **P1 (High):** Unicode filenames, RTL layout, subtitle API, cross-platform consistency
- 3. **P2 (Medium):** Emoji support, Chinese/Japanese text, bi-directional text
- 4. **P3 (Low):** Rare script support (Thai, Devanagari, Greek), TUI rendering

9.3 QA Automation Strategy

- 1. Use Tesseract for deterministic OCR validation (not LLM-dependent)
- 2. Use LLMs for semantic validation (“does this translation make sense?”)
- 3. Use multi-provider consensus for ambiguous cases (pkg/llm/consensus.go)
- 4. Use Groq for rapid API regression (fast + cheap)
- 5. Use Anthropic for visual validation (highest accuracy)
- 6. Track cost per i18n test suite to optimize provider selection

End of Analysis